

Informe Tarea 1

Esteban Castro, ecastroa@usm.cl
Benjamin Margulis, bmargulis@usm.cl
Agustin Villalobos, avillalobos@usm.cl

Parte 1

1.1. Diseño de la función lecturaSensor()

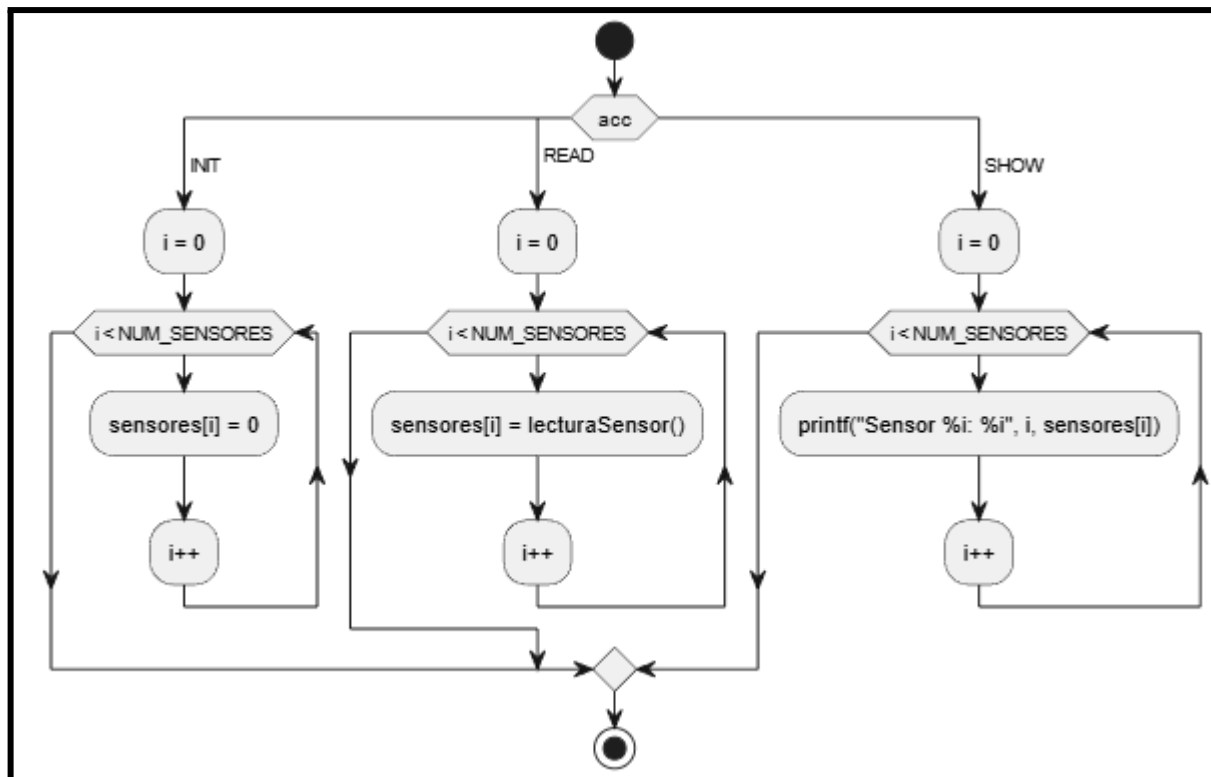
Esta función simula la obtención de datos de hardware. Su objetivo es generar un valor de voltaje simulado.

Requisitos operacionales:

1. Debe entregar un valor numérico dentro del rango de 8 bits (0 a 255).
2. Debe ser aleatoria para simular variaciones reales de un entorno físico.

Justificación del enum Acción: El uso de enum mejora la **legibilidad** y **mantenibilidad** del código. En lugar de usar números como (0, 1, 2) que pueden confundir al programador, usamos etiquetas como (INIT, READ, SHOW). Esto permite que la función usarSensores sea una interfaz única que centraliza todas las operaciones del hardware, facilitando la depuración.

Diagrama de Flujo (Lógica): (figura 1)



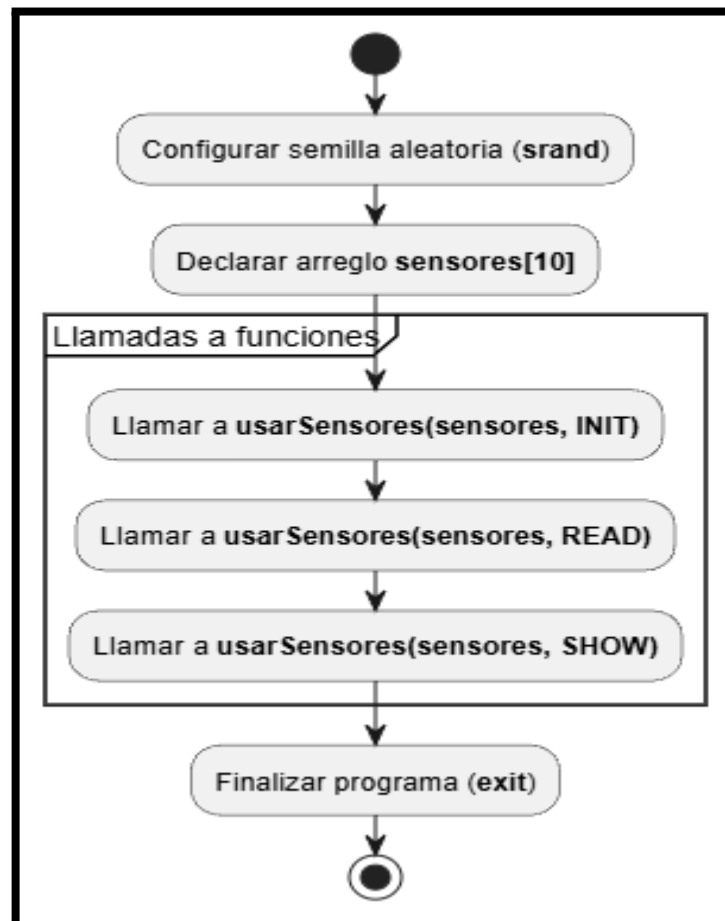
1.2. Diseño completo de la nueva función main()

Explicación: El nuevo main actúa como un orquestador.

1. Primero, inicializa la semilla de aleatoriedad (srand).
2. Define el arreglo de datos.
3. En lugar de tener múltiples bucles for dispersos, realiza tres llamadas secuenciales a usarSensores pasando el arreglo y la acción correspondiente.

Ventaja: Esto desacopla la lógica de control (el "qué hacer") de la implementación técnica (el "cómo se hace").

Diagrama de Flujo (Lógica): (figura 2)



Parte 2

2.1 Hito 1

Qué es lo que retorna la función `clock()`:

Retorna el tiempo de CPU consumido por el programa desde que se inició el proceso hasta el momento de la llamada. No es la hora actual, sino un contador de "tics" de procesador.

Qué unidades tienen las variables `inicio` y `fin`:

Tienen unidades de "**clock ticks**" (pulsos de reloj). No son segundos ni milisegundos de forma directa, sino unidades propias de la implementación del sistema representadas por el tipo `clock_t`.

Por qué se debe dividir la diferencia entre `fin` e `inicio` con el valor constante

`CLOCKS_PER_SEC`:

Porque para convertir los "tics" de procesador a tiempo humano (segundos), necesitamos saber cuántos tics ocurren en un segundo. Esta constante define esa relación. La operación $(\text{tick} * \text{sticks}) / \text{segundo}$ resulta en segundos.

Qué hace `(double)` como prefijo de `(fin - inicio)`:

Es un **casting** o conversión explícita de tipo. Fuerza a que el resultado de la resta (que originalmente es un entero de tipo `clock_t`) sea tratado como un número de punto flotante.

Para qué sirve utilizar `(double)` como prefijo de `(fin - inicio)`:

Sirve para evitar la **división entera**. En C, si divides un entero por otro entero, el resultado será un entero (truncando los decimales). Al convertir el numerador a `double`, forzamos a que la división sea decimal, permitiendo obtener la precisión necesaria para medir fracciones de segundo.

2.2 Hito 2

Explicación en lenguaje natural:

La función implementa un experimento controlado. Recibe un parámetro `caso` que define la carga de trabajo. Se utiliza un bucle `for` de 1.000.000 de iteraciones para que el tiempo

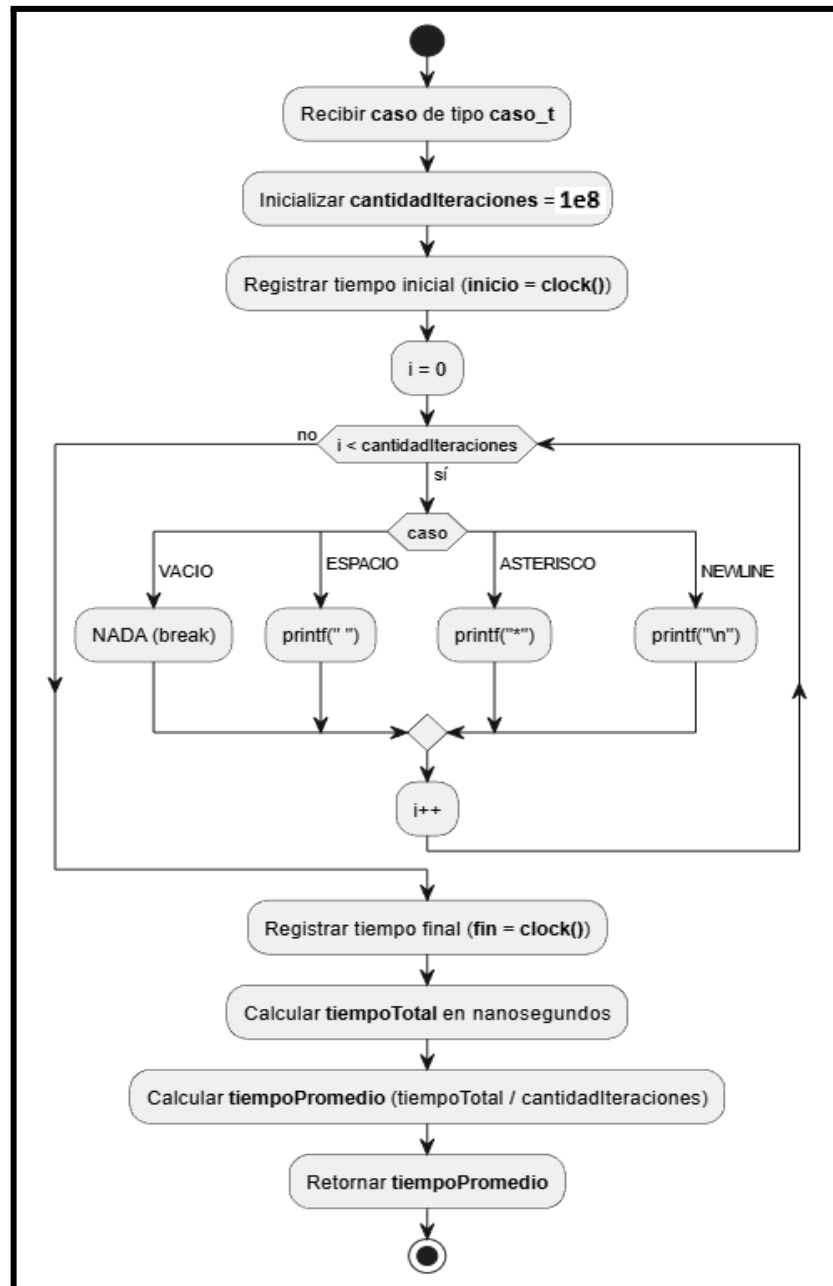
acumulado sea medible. Dentro del bucle, un switch ejecuta la instrucción correspondiente:

VACIO: No hace nada (solo el costo del bucle).

ESPACIO, ASTERISCO, NEWLINE: Realizan una llamada a printf.

Finalmente, calcula el tiempo total, lo convierte a nanosegundos multiplicando por 10^9 y divide por la cantidad de iteraciones para obtener el tiempo unitario.

Diagrama de flujo: (figura 3)



2.3 Hito 3

Para las mediciones se utilizó un script que ejecutó el programa compilado con **GCC versión 15.2.0 (MSYS2)**. un total de 10 veces. La función cicloPrueba se configuró con **1e8 iteraciones** para asegurar que el tiempo de ejecución fuera lo suficientemente largo como para ser capturado con precisión por clock(). .

Características del computador:

- **Procesador:** AMD Ryzen 3400G
- **Memoria RAM:** 16gb Ram
- **SO:** Windows 11.

Tabla de Resultados

Caso	t1	t2		t10	Promedio [ns]	Desv. Estándar
VACIO	5,52	6,01	..	4,92	5,81	0,14
ESPACIO	135,22	148,60	..	143,41	144,41	4,54
ASTERISCO	139,22	129,04	..	137,95	140,74	5,47
NEWLINE	140,71	156,81	..	157,71	151,51	5,08

Análisis de los resultados:

1. VACIO vs Otros:

El caso VACIO presenta un tiempo promedio de solo 5,81 ns, muy inferior al resto de los casos. Esto indica que el ciclo for por sí solo tiene un costo extremadamente bajo cuando no se realizan operaciones de salida. La diferencia respecto a los demás casos demuestra que las operaciones de Entrada/Salida (printf) consumen una cantidad de tiempo mucho mayor que las operaciones internas de la CPU.

2. ESPACIO vs ASTERISCO:

Los casos ESPACIO y ASTERISCO tienen promedios muy similares, de 144,41 ns y 150,74 ns respectivamente. Esto ocurre porque ambos realizan la impresión de un único carácter en pantalla, por lo que el sistema operativo y la terminal manejan prácticamente la misma carga de trabajo. La pequeña diferencia observada puede atribuirse a variaciones normales de ejecución y planificación del sistema.

3. Costo del NEWLINE:

El caso NEWLINE obtuvo el mayor tiempo promedio, con 151,51 ns. Esto se debe a que el carácter `\n` no solo imprime un salto de línea, sino que además puede forzar el vaciado del buffer de salida (*buffer flush*). Como consecuencia, la terminal debe actualizar físicamente la pantalla con mayor frecuencia, aumentando la latencia de la operación.

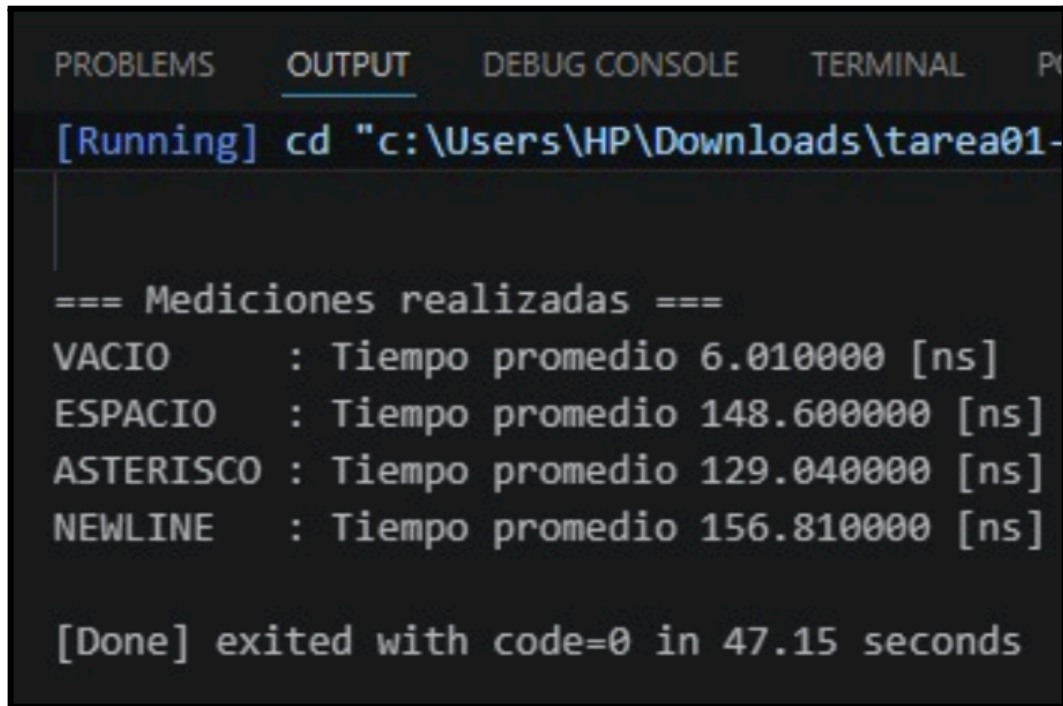
4. Desviación estándar:

El caso VACIO posee la menor desviación estándar (0,14 ns), lo que indica que las mediciones fueron muy consistentes y estables. En cambio, ESPACIO, ASTERISCO y NEWLINE presentan desviaciones estándar mayores, entre 4 y 5 ns aproximadamente. Esto refleja que las operaciones de impresión dependen de factores externos como la gestión de buffers, el sistema operativo y la carga momentánea del computador, generando una variabilidad más alta en los tiempos medidos.

5. Conclusión general:

Los resultados permiten concluir que las operaciones de impresión en consola son significativamente más costosas que las operaciones internas del procesador. Además, el uso de `\n` introduce un costo adicional debido al manejo del buffer de salida y la actualización de la terminal.

constancia del código corriendo (t2): (figura 4)



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  P
[Running] cd "c:\Users\HP\Downloads\tarea01-

=== Mediciones realizadas ===
VACIO      : Tiempo promedio 6.010000 [ns]
ESPACIO    : Tiempo promedio 148.600000 [ns]
ASTERISCO  : Tiempo promedio 129.040000 [ns]
NEWLINE    : Tiempo promedio 156.810000 [ns]

[Done] exited with code=0 in 47.15 seconds
```